

Lecture 5

Multiple Linear Regression; Feature Engineering, Gradient Descent

DSC 40A, Summer 2026

Announcements at break

Agenda

- Multiple linear regression.
- Interpreting parameters.
- Feature engineering and transformations.

Question 🤔

Answer in chat

Remember, you can always ask questions in Q&A!

Multiple linear regression

	departure_hour	day_of_month	minutes
0	10.816667	15	68.0
1	7.750000	16	94.0
2	8.450000	22	63.0
3	7.133333	23	100.0
4	9.150000	30	69.0
...	...	...	...

So far, we've fit **simple** linear regression models, which use only **one** feature (`'departure_hour'`) for making predictions.

Incorporating multiple features

- In the context of the commute times dataset, the simple linear regression model we fit was of the form:

$$\begin{aligned}\text{pred. commute} &= H(\text{departure hour}) \\ &= w_0 + w_1 \cdot \text{departure hour}\end{aligned}$$

- Now, we'll try and fit a multiple linear regression model of the form:

$$\begin{aligned}\text{pred. commute} &= H(\text{departure hour}) \\ &= w_0 + w_1 \cdot \text{departure hour} + w_2 \cdot \text{day of month}\end{aligned}$$

- Linear regression with **multiple** features is called **multiple linear regression**.
- How do we find w_0^* , w_1^* , and w_2^* ?

Geometric interpretation

- The hypothesis function:

$$H(\text{departure hour}) = w_0 + w_1 \cdot \text{departure hour}$$

looks like a **line** in 2D.

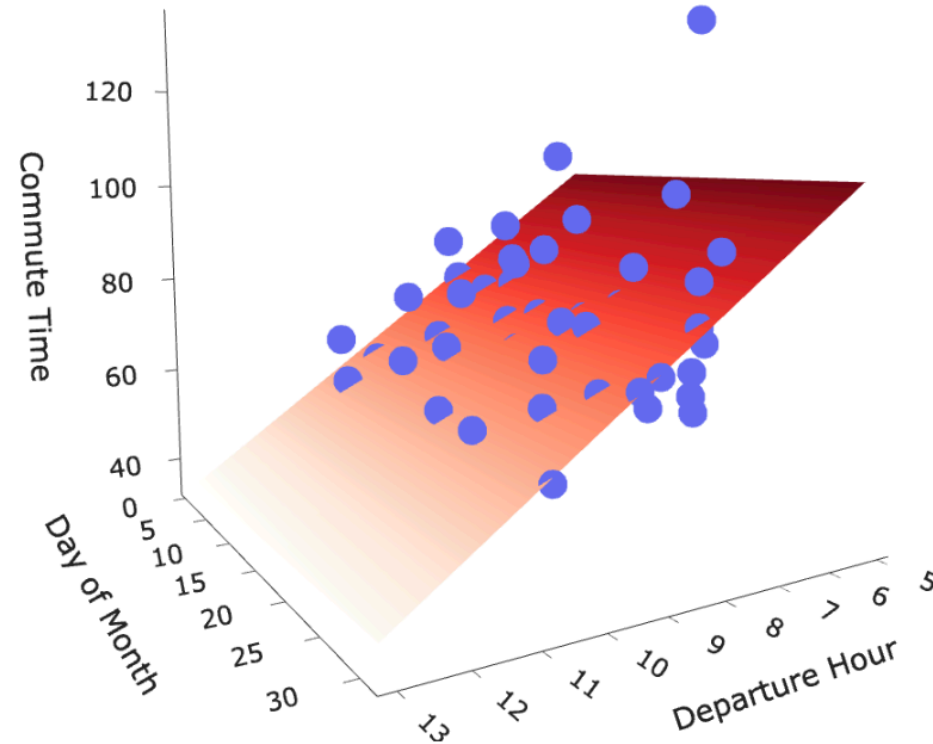
- **Questions:**

- How many dimensions do we need to graph the hypothesis function:

$$H(\text{departure hour}) = w_0 + w_1 \cdot \text{departure hour} + w_2 \cdot \text{day of month}$$

- What is the shape of the hypothesis function?

Commute Time vs. Departure Hour and Day of Month



Our new hypothesis function is a **plane** in 3D!

Our goal is to find the **plane** of best fit that pierces through the cloud of points.

The setup

- Suppose we have the following dataset.

	departure_hour	day_of_month	minutes
row			
1	8.45	22	63.0
2	8.90	28	89.0
3	8.72	18	89.0

- We can represent each day with a **feature vector**, $\vec{x}$:

The hypothesis vector

- When our hypothesis function is of the form:

$$H(\text{departure hour}) = w_0 + w_1 \cdot \text{departure hour} + w_2 \cdot \text{day of month}$$

the hypothesis vector $\vec{h} \in \mathbb{R}^n$ can be written as:

$$\vec{h} = \begin{bmatrix} H(\text{departure hour}_1, \text{day}_1) \\ H(\text{departure hour}_2, \text{day}_2) \\ \dots \\ H(\text{departure hour}_n, \text{day}_n) \end{bmatrix} = \begin{bmatrix} 1 & \text{departure hour}_1 & \text{day}_1 \\ 1 & \text{departure hour}_2 & \text{day}_2 \\ \dots & \dots & \dots \\ 1 & \text{departure hour}_n & \text{day}_n \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

Finding the optimal parameters

- To find the optimal parameter vector, $\vec{w}^*$, we can use the **design matrix** $X \in \mathbb{R}^{n \times 3}$ and **observation vector** $\vec{y} \in \mathbb{R}^n$:

$$X = \begin{bmatrix} 1 & \text{departure hour}_1 & \text{day}_1 \\ 1 & \text{departure hour}_2 & \text{day}_2 \\ \dots & \dots & \dots \\ 1 & \text{departure hour}_n & \text{day}_n \end{bmatrix} \quad \vec{y} = \begin{bmatrix} \text{commute time}_1 \\ \text{commute time}_2 \\ \vdots \\ \text{commute time}_n \end{bmatrix}$$

- Then, all we need to do is solve the **normal equations**:

$$X^T X \vec{w}^* = X^T \vec{y}$$

If $X^T X$ is invertible, we know the solution is:

$$\vec{w}^* = (X^T X)^{-1} X^T \vec{y}$$

Notation for multiple linear regression

- We will need to keep track of multiple features for every individual in our dataset.
 - In practice, we could have hundreds or thousands of features!
- As before, subscripts distinguish between individuals in our dataset. We have n individuals, also called **training examples**.
- Superscripts distinguish between **features**. We have d features.

departure hour: $x^{(1)}$

day of month: $x^{(2)}$

Think of $x^{(1)}$, $x^{(2)}$, ... as new variable names, like new letters.

The hypothesis vector

- When our hypothesis function is of the form:

$$H(\text{departure hour}) = w_0 + w_1 \cdot \text{departure hour} + w_2 \cdot \text{day of month}$$

the hypothesis vector $\vec{h} \in \mathbb{R}^n$ can be written as:

$$\vec{h} = \begin{bmatrix} H(\text{departure hour}_1, \text{day}_1) \\ H(\text{departure hour}_2, \text{day}_2) \\ \dots \\ H(\text{departure hour}_n, \text{day}_n) \end{bmatrix} = \begin{bmatrix} 1 & \text{departure hour}_1 & \text{day}_1 \\ 1 & \text{departure hour}_2 & \text{day}_2 \\ \dots & \dots & \dots \\ 1 & \text{departure hour}_n & \text{day}_n \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

Augmented feature vectors

- The **augmented feature vector** $\text{Aug}(\vec{x})$ is the vector obtained by adding a 1 to the front of feature vector $\vec{x}$:

$$\vec{x} = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(d)} \end{bmatrix} \quad \text{Aug}(\vec{x}) = \begin{bmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(d)} \end{bmatrix} \quad \vec{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

- Then, our hypothesis function is:

$$\begin{aligned} H(\vec{x}) &= w_0 + w_1 x^{(1)} + w_2 x^{(2)} + \dots + w_d x^{(d)} \\ &= \vec{w} \cdot \text{Aug}(\vec{x}) \end{aligned}$$

The general problem

- We have n data points, $(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_n, y_n)$, where each $\vec{x}_i$ is a feature vector of d features:

$$\vec{x}_i = \begin{bmatrix} x_i^{(1)} \\ x_i^{(2)} \\ \vdots \\ x_i^{(d)} \end{bmatrix}$$

- We want to find a good linear hypothesis function:

$$\begin{aligned} H(\vec{x}) &= w_0 + w_1 x^{(1)} + w_2 x^{(2)} + \dots + w_d x^{(d)} \\ &= \vec{w} \cdot \text{Aug}(\vec{x}) \end{aligned}$$

The general solution

- Define the **design matrix** $X \in \mathbb{R}^{n \times (d+1)}$ and **observation vector** $\vec{y} \in \mathbb{R}^n$:

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(d)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(d)} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n^{(1)} & x_n^{(2)} & \dots & x_n^{(d)} \end{bmatrix} = \begin{bmatrix} \text{Aug}(\vec{x}_1)^T \\ \text{Aug}(\vec{x}_2)^T \\ \vdots \\ \text{Aug}(\vec{x}_n)^T \end{bmatrix} \quad \vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

- Then, solve the **normal equations** to find the optimal parameter vector, $\vec{w}^*$:

$$X^T X \vec{w}^* = X^T \vec{y}$$

Terminology for parameters

- With d features, $\vec{w}$ has $d + 1$ entries.
- w_0 is the **bias**, also known as the **intercept**.
- $w_1, w_2, \dots, w_d$ each give the **weight**, or **coefficient**, or **slope**, of a feature.

$$H(\vec{x}) = w_0 + w_1 x^{(1)} + w_2 x^{(2)} + \dots + w_d x^{(d)}$$

Interpreting parameters

Example: Predicting sales

- For each of 26 stores, we have:
 - net sales,
 - square feet,
 - inventory,
 - advertising expenditure,
 - district size, and
 - number of competing stores.

- **Goal:** Predict net sales given the other five features.
- To begin, we'll start trying to fit the hypothesis function to predict sales:

$$H(\text{square feet, competitors}) = w_0 + w_1 \cdot \text{square feet} + w_2 \cdot \text{competitors}$$

Question 🤔

Answer in chat

$$H(\text{square feet, competitors}) = w_0 + w_1 \cdot \text{square feet} + w_2 \cdot \text{competitors}$$

What will be the signs of w_1^* and w_2^* ?

- A. $w_1^* +$ $w_2^* +$
- B. $w_1^* +$ $w_2^* -$
- A. $w_1^* -$ $w_2^* +$
- A. $w_1^* -$ $w_2^* -$

Let's find out! Follow along in [this notebook](#).

Question 🤔

Answer in chat

Which feature is most "important"?

- A. square feet: $w_1^* = 16.202$
- B. competitors: $w_2^* = -5.311$
- C. inventory: $w_3^* = 0.175$
- D. advertising: $w_4^* = 11.526$
- E. district size: $w_5^* = 13.580$

Which features are most "important"?

- The most important feature is **not necessarily** the feature with largest magnitude weight.
- Features are measured in different units, i.e. different scales.
 - Suppose I fit one hypothesis function, H_1 , with sales in US dollars, and another hypothesis function, H_2 , with sales in Japanese yen (1 USD $\approx$ 157 yen).
 - Sales is just as important in both hypothesis functions.
 - But the weight of sales in H_1 will be 157 times larger than the weight of sales in H_2 .
- **Solution:** If you care about the interpretability of the resulting weights, **standardize** each feature before performing regression, i.e. convert each feature to standard units.

Standard units

- Recall: to convert a feature $x_1, x_2, \dots, x_n$ to standard units, we use the formula:

$$x_i \text{ (su)} = \frac{x_i - \bar{x}}{\sigma_x}$$

- Example: 1, 7, 7, 9.

- Mean: $\frac{1+7+7+9}{4} = \frac{24}{4} = 6$.

- Standard deviation:

$$\text{SD} = \sqrt{\frac{1}{4}((1-6)^2 + (7-6)^2 + (7-6)^2 + (9-6)^2)} = \sqrt{\frac{1}{4} \cdot 36} = 3$$

- Standardized data:

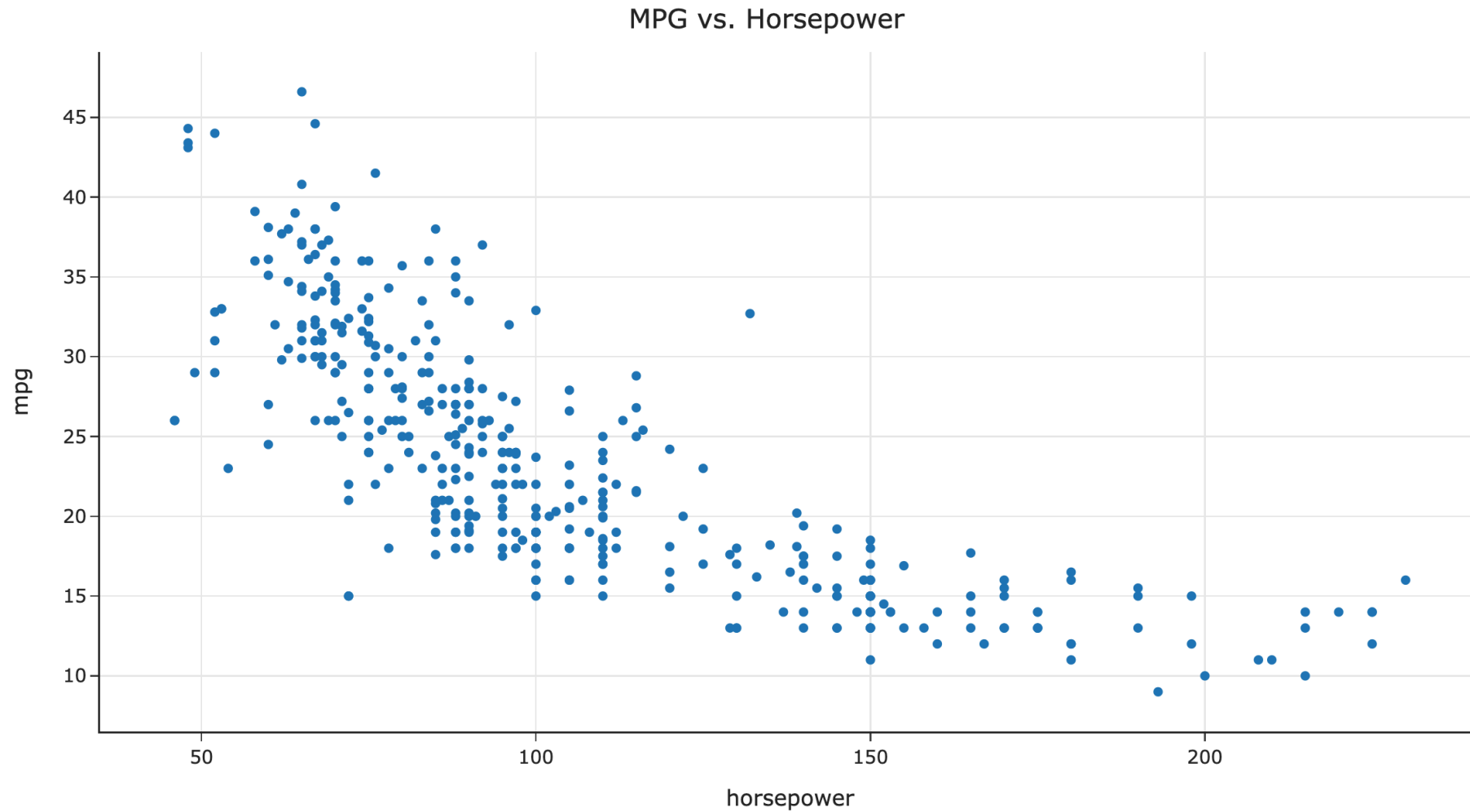
$$1 \mapsto \frac{1-6}{3} = \boxed{-\frac{5}{3}} \quad 7 \mapsto \frac{7-6}{3} = \boxed{\frac{1}{3}} \quad 7 \mapsto \boxed{\frac{1}{3}} \quad 9 \mapsto \frac{9-6}{3} = \boxed{1}$$

Standard units for multiple linear regression

- The result of standardizing each feature (separately!) is that the units of each feature are on the same scale.
 - There's no need to standardize the outcome (net sales), since it's not being compared to anything.
 - Also, we can't standardize the column of all 1s.
- Then, solve the normal equations. The resulting $w_0^*, w_1^*, \dots, w_d^*$ are called the **standardized regression coefficients**.
- Standardized regression coefficients can be directly compared to one another.
- Note that standardizing each feature **does not** change the MSE of the resulting hypothesis function!

Once again, let's try it out! Follow along in [this notebook](#).

Feature engineering and transformations



Question: Would a linear hypothesis function work well on this dataset?

A quadratic hypothesis function

- It looks like there's some sort of quadratic relationship between horsepower and MPG in the last scatter plot. We want to try and fit a hypothesis function of the form:

$$H(x) = w_0 + w_1x + w_2x^2$$

- Note that while this is quadratic in horsepower, it is **linear in the parameters!**
 - That is, it is a **linear combination of features**.
- We can do that, by choosing our two "features" to be x_i and x_i^2 , respectively.
 - In other words, $x_i^{(1)} = x_i$ and $x_i^{(2)} = x_i^2$.
 - More generally, we can create new features out of existing features.

A quadratic hypothesis function

- Desired hypothesis function: $H(x) = w_0 + w_1x + w_2x^2$.
- The resulting design matrix looks like:

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \dots & & \\ 1 & x_n & x_n^2 \end{bmatrix}$$

- To find the optimal parameter vector $\vec{w}^*$, we need to solve the **normal equations!**

$$X^T X \vec{w}^* = X^T \vec{y}$$

More examples

- What if we want to use a hypothesis function of the form:

$$H(x) = w_0 + w_1x + w_2x^2 + w_3x^3?$$

- What if we want to use a hypothesis function of the form:

$$H(x) = w_1 \frac{1}{x^2} + w_2 \sin x + w_3 e^x?$$

Feature engineering

- The process of creating new features out of existing information in our dataset is called **feature engineering**.
- In this class, feature engineering will mostly be restricted to creating non-linear functions of existing features (as in the previous example).
- In the future you'll learn how to do other things, like encode categorical information.

Non-linear functions of multiple features

- Recall our earlier example of predicting sales from square footage and number of competitors. What if we want a hypothesis function of the form:

$$\begin{aligned} H(\text{sqft}, \text{comp}) &= w_0 + w_1 \cdot \text{sqft} + w_2 \cdot \text{sqft}^2 + w_3 \cdot \text{comp} + w_4 \cdot (\text{sqft} \cdot \text{comp}) \\ &= w_0 + w_1 s + w_2 s^2 + w_3 c + w_4 sc \end{aligned}$$

- The solution is to choose a design matrix accordingly:

$$X = \begin{bmatrix} 1 & s_1 & s_1^2 & c_1 & s_1 c_1 \\ 1 & s_2 & s_2^2 & c_2 & s_2 c_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & s_n & s_n^2 & c_n & s_n c_n \end{bmatrix}$$

Finding the optimal parameter vector, $\vec{w}^*$

- As long as the form of the hypothesis function permits us to write $\vec{h} = X\vec{w}$ for some X and $\vec{w}$, the mean squared error is:

$$R_{\text{sq}}(\vec{w}) = \frac{1}{n} \|\vec{y} - X\vec{w}\|^2$$

- Regardless of the values of X and $\vec{y}$, the value of $\vec{w}^*$ that minimizes $R_{\text{sq}}(\vec{w})$ is the solution to the **normal equations**:

$$X^T X \vec{w}^* = X^T \vec{y}$$

Linear in the parameters

- We can fit rules like:

$$w_0 + w_1x + w_2x^2 \quad w_1e^{-x^{(1)2}} + w_2 \cos(x^{(2)} + \pi) + w_3 \frac{\log 2x^{(3)}}{x^{(2)}}$$

- This includes arbitrary polynomials.
- These are all linear combinations of (just) features.

- We can't fit rules like:

$$w_0 + e^{w_1x} \quad w_0 + \sin(w_1x^{(1)} + w_2x^{(2)})$$

- These are **not** linear combinations of just features!
- We can have any number of parameters, as long as our hypothesis function is **linear in the parameters**, or linear when we think of it as a function of the parameters.

Determining function form

- How do we know what form our hypothesis function should take?
- Sometimes, we know from *theory*, using knowledge about what the variables represent and how they should be related.
- Other times, we make a guess based on the data.
- Generally, start with simpler functions first.
 - Remember, the goal is to find a hypothesis function that will generalize well to unseen data.

Example: Amdahl's Law

- Amdahl's Law relates the runtime of a program on p processors to the time to do the sequential and nonsequential parts on one processor.

$$H(p) = t_S + \frac{t_{NS}}{p}$$

- Collect data by timing a program with varying numbers of processors:

Processors	Time (Hours)
1	8
2	4
4	3

Example: Fitting $H(x) = w_0 + w_1 \cdot \frac{1}{x}$

Processors	Time (Hours)
1	8
2	4
4	3

How do we fit hypothesis functions that aren't linear in the parameters?

- Suppose we want to fit the hypothesis function:

$$H(x) = w_0 e^{w_1 x}$$

- This is **not** linear in terms of w_0 and w_1 , so our results for linear regression don't apply.
- **Possible solution:** Try to apply a **transformation**.

Transformations

- **Question:** Can we re-write $H(x) = w_0 e^{w_1 x}$ as a hypothesis function that **is** linear in the parameters?

Transformations

- **Solution:** Create a new hypothesis function, $T(x)$, with parameters b_0 and b_1 , where $T(x) = b_0 + b_1x$.
- This hypothesis function is related to $H(x)$ by the relationship $T(x) = \log H(x)$.
- $\vec{b}$ is related to $\vec{w}$ by $b_0 = \log w_0$ and $b_1 = w_1$.
- Our new observation vector, $\vec{z}$, is
$$\begin{bmatrix} \log y_1 \\ \log y_2 \\ \dots \\ \log y_n \end{bmatrix}.$$
- $T(x) = b_0 + b_1x$ is linear in its parameters, b_0 and b_1 .
- Use the solution to the normal equations to find $\vec{b}^*$, and the relationship between $\vec{b}$ and $\vec{w}$ to find $\vec{w}^*$.

Once again, let's try it out! Follow along in [this notebook](#).

Non-linear hypothesis functions in general

- Sometimes, it's just not possible to transform a hypothesis function to be linear in terms of some parameters.
- In those cases, you'd have to resort to other methods of finding the optimal parameters.

- For example, $H(x) = w_0 \sin(w_1 x)$ **can't** be transformed to be linear.
- But, there are other methods of minimizing mean squared error:

$$R_{\text{sq}}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (y_i - w_0 \sin(w_1 x))^2$$

- One method: **gradient descent**, the topic of the next lecture!
- Hypothesis functions that are linear in the parameters are much easier to work with.

Announcements

- Reminder! Community OH today, here, after lecture!
- Homework 4 is due on **Tuesday July 14th**.
 - It should be quick and good prep for the exam.
- Midterm Review session on Tuesday July 14th at 6:00pm PT
 - Will be 50 minutes long.
 - Will not be recorded, but materials will be released.
- Note about today's content on the upcoming midterm

The Midterm Exam is on Wednesday, July 15th!

- Exam Proctoring AND FORMATTING Instructions have been sent out. Read them in full, carefully.
 - It is **your** responsibility to read them and ensure you have a proper proctoring set up. We will not be waiting to start the exam if you are late or are not set up; your exam score will just be canceled.
 - We are being very strict with proctoring for exam security, and any student caught without the correct proctoring set is eligible to having their exam score canceled.
- Many of you are in non-PT time zones. Please plan to take the exam during the expected exam time slot: Wednesday, July 15th, 11:00AM-1:50PM, PT.
 - We will **not** be accommodating different exam times, unless you have OSD accommodations specifically for that.
 - We will be confirming the length of the exam today or tomorrow.

Reminder

- This was expected to be a fast paced course.
- We are covering 10 weeks of content in 5 weeks.
 - Which means you should expect the course to go at double speed.
- We know you are human.
 - We are doing our best to account for that.
 - While you might feel overwhelmed, there is only so much we can do to reduce the amount you need to do while still covering everything that should be covered.
- Homeworks have a tight turnaround
 - The turnaround on homeworks is the exact same it has been for past iterations of this course
 - Homeworks, in a remote course, are the only way to hold students accountable for their learning, other than exams.

Lecture 5 Part 2

Feature Engineering, Gradient Descent

DSC 40A, Summer 2026

Agenda

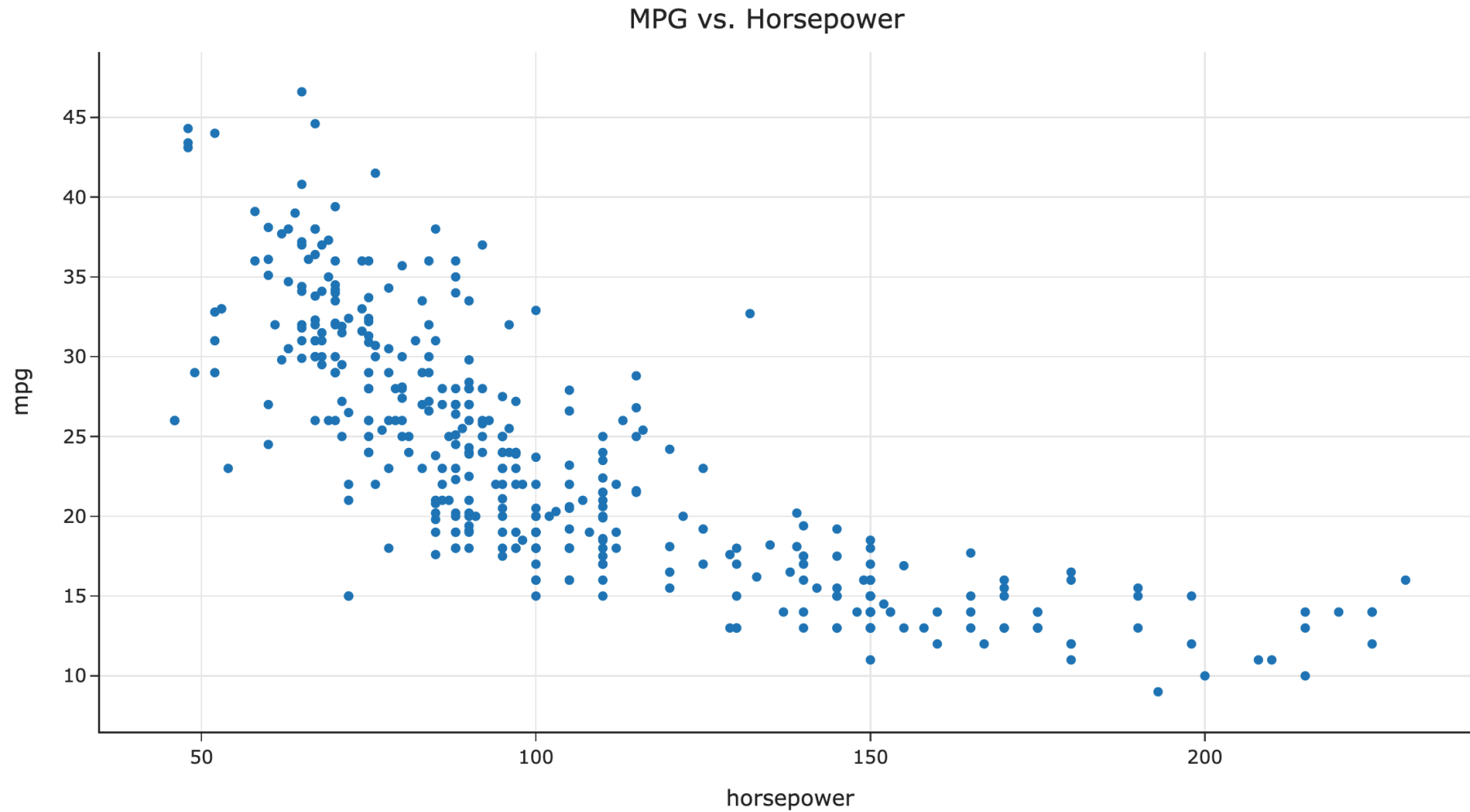
- Feature engineering and transformations.
- Minimizing functions using gradient descent.

Question 🤔

Answer in chat

Remember, you can always ask questions in Q&A!

Feature engineering and transformations



Question: Would a linear hypothesis function work well on this dataset?

Linear in the parameters

- We can fit rules like:

$$w_0 + w_1x + w_2x^2 \quad w_1e^{-x^{(1)2}} + w_2 \cos(x^{(2)} + \pi) + w_3 \frac{\log 2x^{(3)}}{x^{(2)}}$$

- This includes arbitrary polynomials.
- These are all linear combinations of (just) features.

- We can't fit rules like:

$$w_0 + e^{w_1x} \quad w_0 + \sin(w_1x^{(1)} + w_2x^{(2)})$$

- These are **not** linear combinations of just features!
- We can have any number of parameters, as long as our hypothesis function is **linear in the parameters**, or linear when we think of it as a function of the parameters.

How do we fit hypothesis functions that aren't linear in the parameters?

Possible solution: Try to apply a transformation.

Transformations

- **Question:** Can we re-write $H(x) = w_0 e^{w_1 x}$ as a hypothesis function that is linear in the parameters?

Non-linear hypothesis functions in general

- Sometimes, it's just not possible to transform a hypothesis function to be linear in terms of some parameters.
- In those cases, you'd have to resort to other methods of finding the optimal parameters.

- For example, $H(x) = w_0 \sin(w_1 x)$ **can't** be transformed to be linear.
- But, there are other methods of minimizing mean squared error:

$$R_{\text{sq}}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (y_i - w_0 \sin(w_1 x))^2$$

- One method: **gradient descent**, the topic we're going to look at next!
- Hypothesis functions that are linear in the parameters are much easier to work with.

Question 🤔

Answer in chat

Which hypothesis function is **not** linear in the parameters?

- A. $H(\vec{x}) = w_1(x^{(1)}x^{(2)}) + \frac{w_2}{x^{(1)}}\sin(x^{(2)})$
- B. $H(\vec{x}) = 2^{w_1}x^{(1)}$
- C. $H(\vec{x}) = \vec{w} \cdot \text{Aug}(\vec{x})$
- D. $H(\vec{x}) = w_1 \cos(x^{(1)}) + w_2 2^{x^{(2)}} \log x^{(3)}$
- E. More than one of the above.

The modeling recipe

1. Choose a model.
2. Choose a loss function.
3. Minimize average loss to find optimal model parameters.

Minimizing functions using gradient descent

Minimizing empirical risk

- Repeatedly, we've been tasked with **minimizing** the value of empirical risk functions.
 - Why? To help us find the **best** model parameters, h^* or w^* , which help us make the **best** predictions!
- We've minimized empirical risk functions in various ways.

- $R_{\text{sq}}(h) = \frac{1}{n} \sum_{i=1}^n (y_i - h)^2$

- $R_{\text{abs}}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n |y_i - (w_0 + w_1 x)|$

- $R_{\text{sq}}(\vec{w}) = \frac{1}{n} \|\vec{y} - X\vec{w}\|^2$

Minimizing arbitrary functions

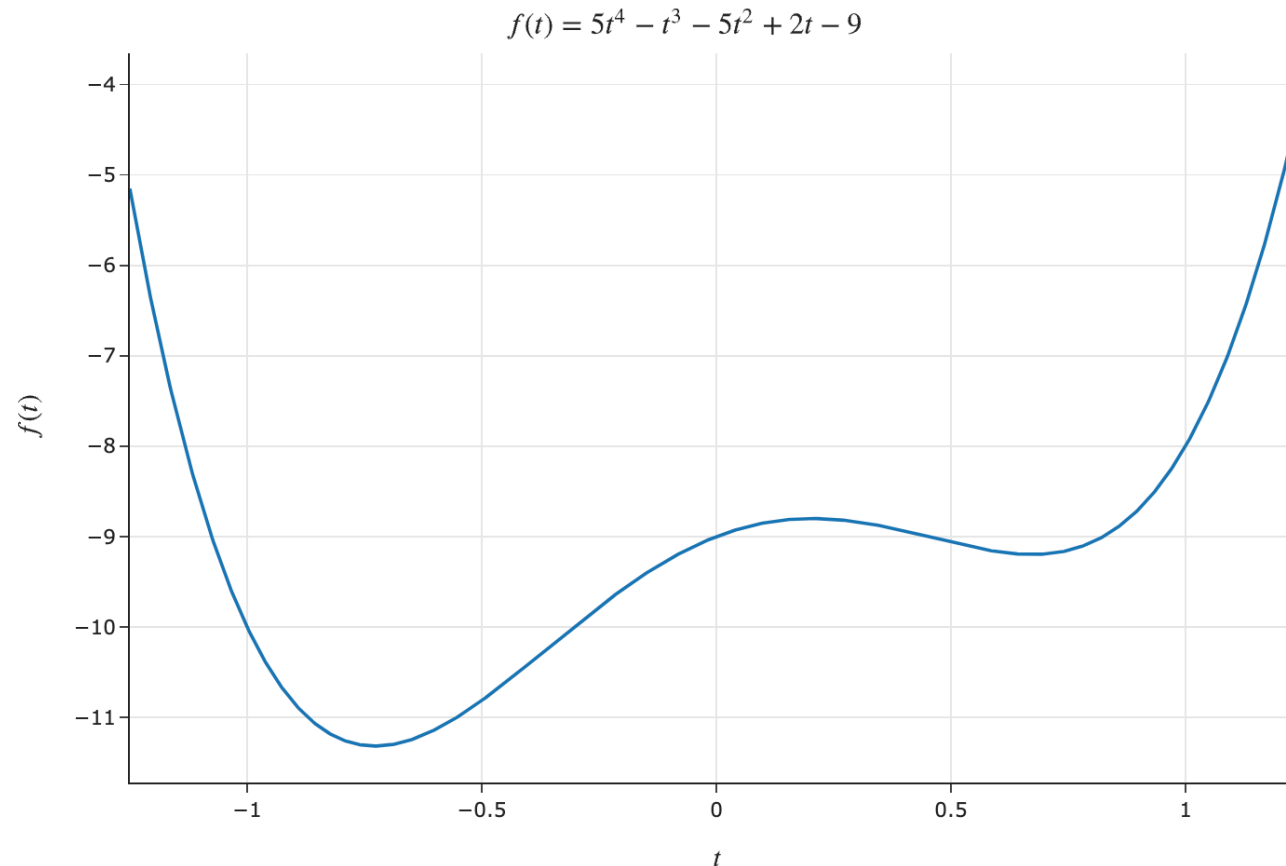
- Assume $f(t)$ is some **differentiable** single-variable function.
- When tasked with minimizing $f(t)$, our general strategy has been to:
 - i. Find $\frac{df}{dt}(t)$, the derivative of f .
 - ii. Find the input t^* such that $\frac{df}{dt}(t^*) = 0$.
- However, there are cases where we can find $\frac{df}{dt}(t)$, but **it is either difficult or impossible to solve** $\frac{df}{dt}(t^*) = 0$.

$$f(t) = 5t^4 - t^3 - 5t^2 + 2t - 9$$

- Then what?

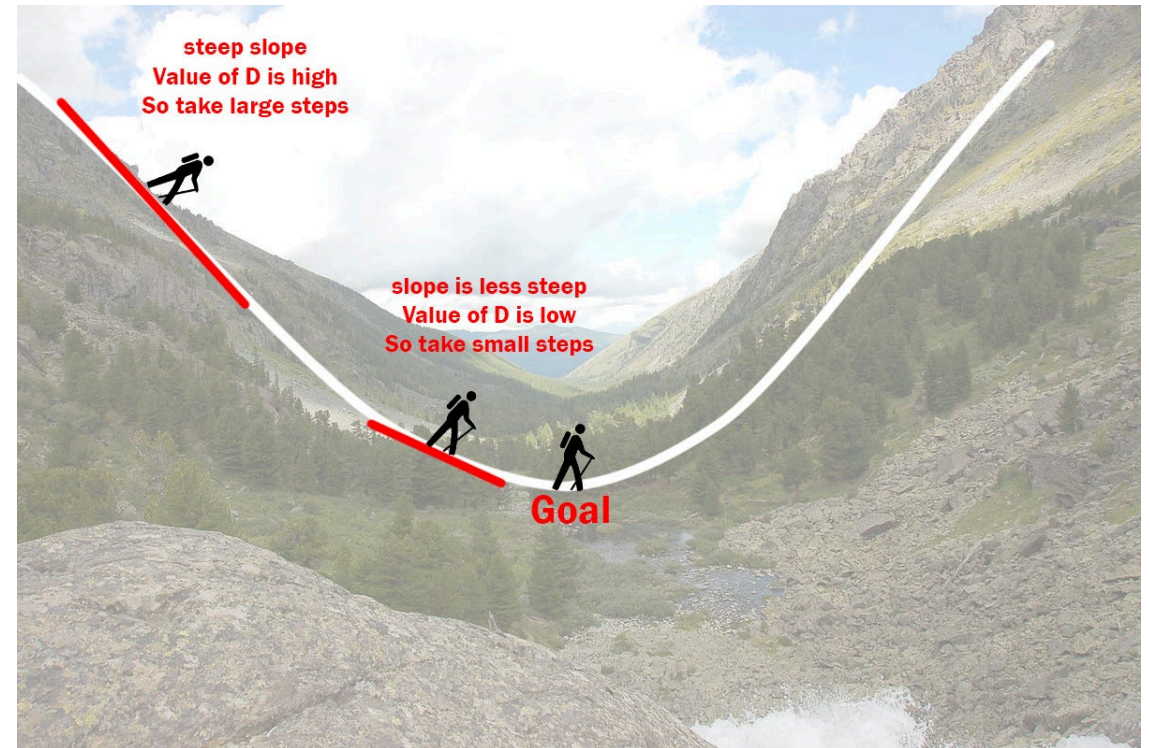
What does the derivative of a function tell us?

- **Goal:** Given a **differentiable** function $f(t)$, find the input t^* that minimizes $f(t)$.
- What does $\frac{d}{dt} f(t)$ mean?

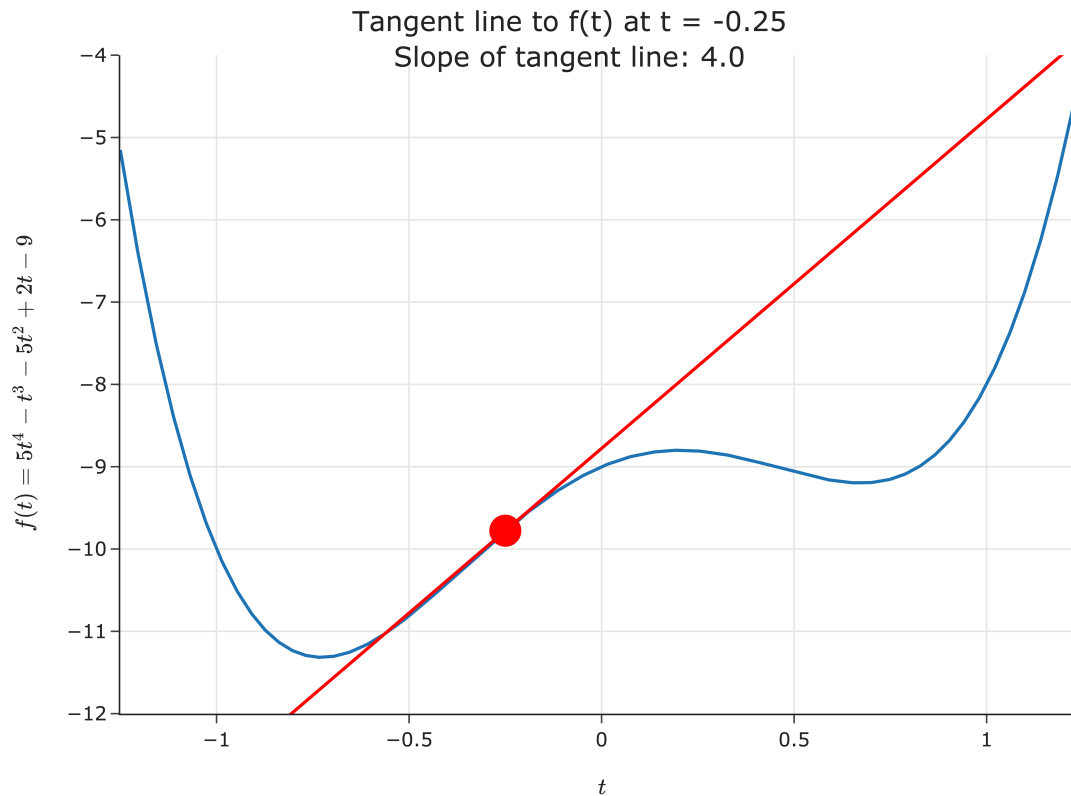


Let's go hiking!

- Suppose you're at the top of a mountain 🏔️ and need to get **to the bottom**.
- Further, suppose it's really cloudy ☁️, meaning you can only see a few feet around you.
- **How** would you get to the bottom?



Searching for the minimum

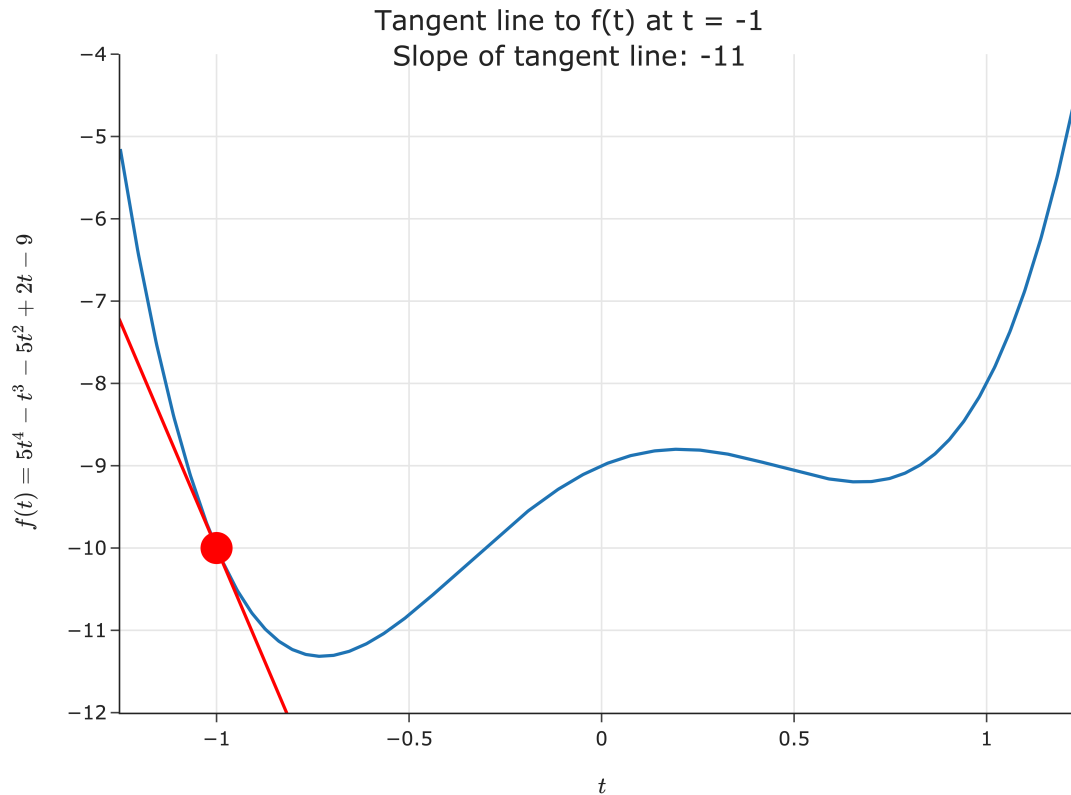


Suppose we're given an initial *guess* for a value of t that minimizes $f(t)$.

If the **slope of the tangent line** at $f(t)$ is **positive** 📈:

- Increasing t **increases** f .
- This means the minimum must be to the **left** of the point $(t, f(t))$.
- Solution: **Decrease** t ⬇️.

Searching for the minimum



Suppose we're given an initial *guess* for a value of t that minimizes $f(t)$.

If the **slope of the tangent line** at $f(t)$ is **negative** 📉:

- Increasing t **decreases** f .
- This means the minimum must be to the **right** of the point $(t, f(t))$.
- Solution: **Increase t** ⬆️.

Intuition

- To minimize $f(t)$, start with an initial guess t_0 .
- Where do we go next?
 - If $\frac{df}{dt}(t_0) > 0$, **decrease** t_0 .
 - If $\frac{df}{dt}(t_0) < 0$, **increase** t_0 .
- One way to accomplish this:

$$t_1 = t_0 - \frac{df}{dt}(t_0)$$

Gradient descent

To minimize a **differentiable** function f :

- Pick a positive number, α . This number is called the **learning rate**, or **step size**.
- Pick an **initial guess**, t_0 .
- Then, repeatedly update your guess using the **update rule**:

$$t_{i+1} = t_i - \alpha \frac{df}{dt}(t_i)$$

- Repeat this process until **convergence** – that is, when t doesn't change much.
- This procedure is called **gradient descent**.

What is gradient descent?

- Gradient descent is a numerical method for finding the input to a function f that minimizes the function.
- Why is it called **gradient** descent?
 - The gradient is the extension of the derivative to functions of multiple variables.
 - We will see how to use gradient descent with multivariate functions next class.
- What is a **numerical** method?
 - A numerical method is a technique for approximating the solution to a mathematical problem, often by using the computer.
- Gradient descent is **widely used** in machine learning, to train models from linear regression to neural networks and transformers (including ChatGPT)!

Let's check out some [animations](#)!

Gradient descent and empirical risk minimization

- While gradient descent can minimize other kinds of differentiable functions, its most common use case is in **minimizing empirical risk**.
- For example, consider:
 - The constant model, $H(x) = h$.
 - The dataset $-4, -2, 2, 4$.
 - The initial guess $h_0 = 4$ and the learning rate $\alpha = \frac{1}{4}$.
- **Exercise:** Find h_1 and h_2 .

Lingering questions

Next, we'll explore the following ideas:

- When is gradient descent *guaranteed* to converge to a global minimum?
 - What kinds of functions work well with gradient descent?
- How do I choose a step size?
- How do I use gradient descent to minimize functions of multiple variables, e.g.:

$$R_{\text{sq}}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (y_i - (w_0 + w_1 x_i))^2$$

Lecture 5 Part 3

Gradient Descent, Continued

DSC 40A, Summer 2026

Agenda

- Recap: Gradient descent.
- Convexity.
- More examples.
 - Huber loss.
 - Gradient descent with multiple variables.

Question 🤔

Answer in chat

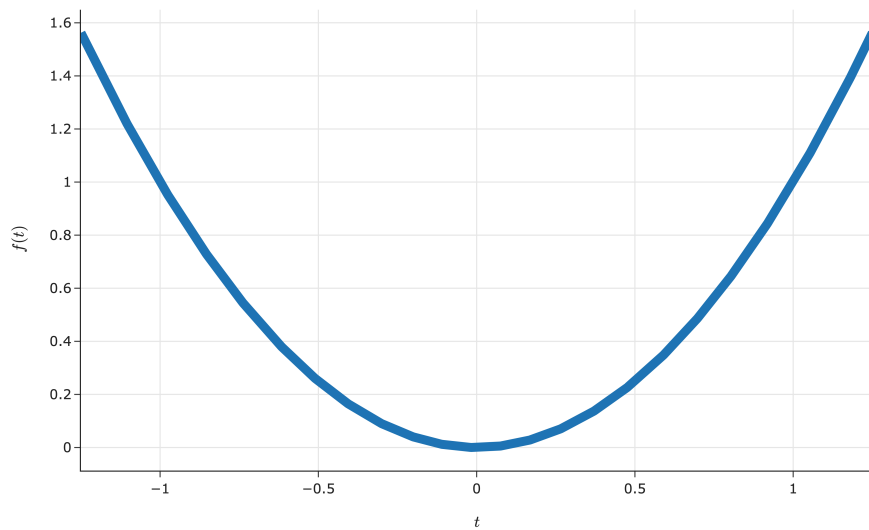
Remember, you can always ask questions in Q&A!

Overview: Gradient descent

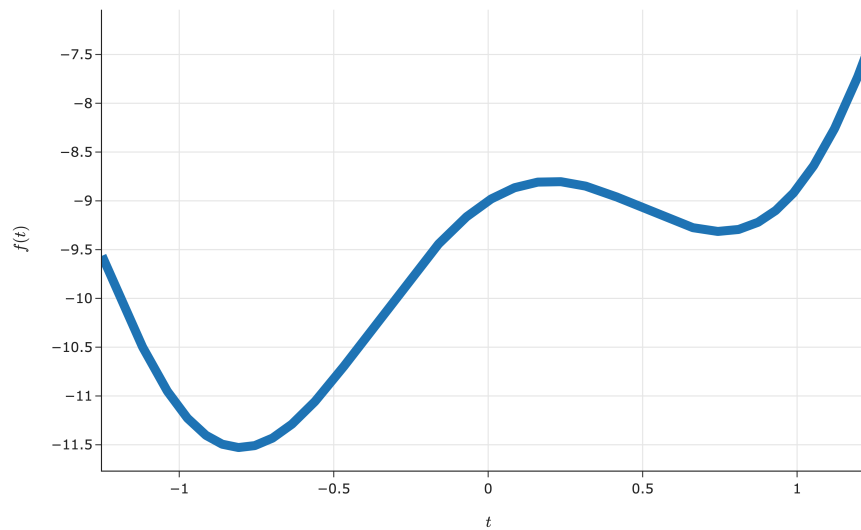
When is gradient descent guaranteed to work?

Convex functions

A convex function ✓



A non-convex function ✗



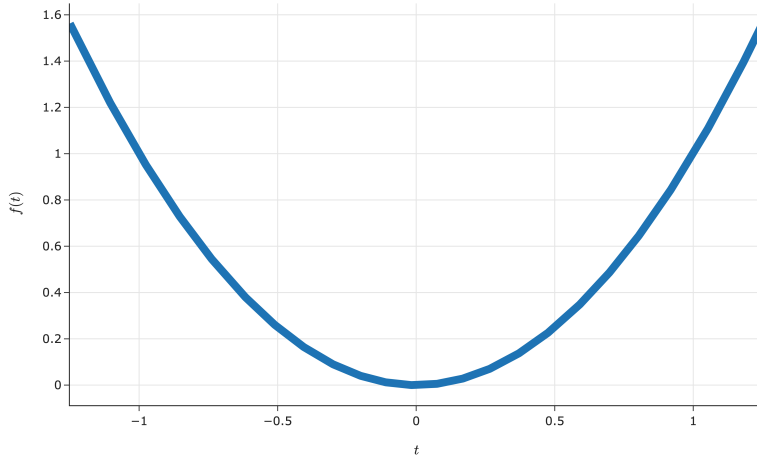
Convexity

- A function f is **convex** if, for **every** a, b in the domain of f , the line segment between:

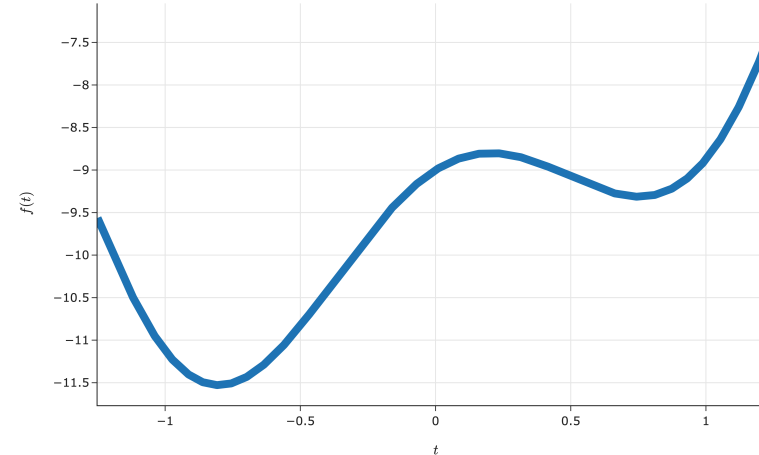
$$(a, f(a)) \text{ and } (b, f(b))$$

does not go below the plot of f .

A **convex** function ✓



A **non-convex** function ✗

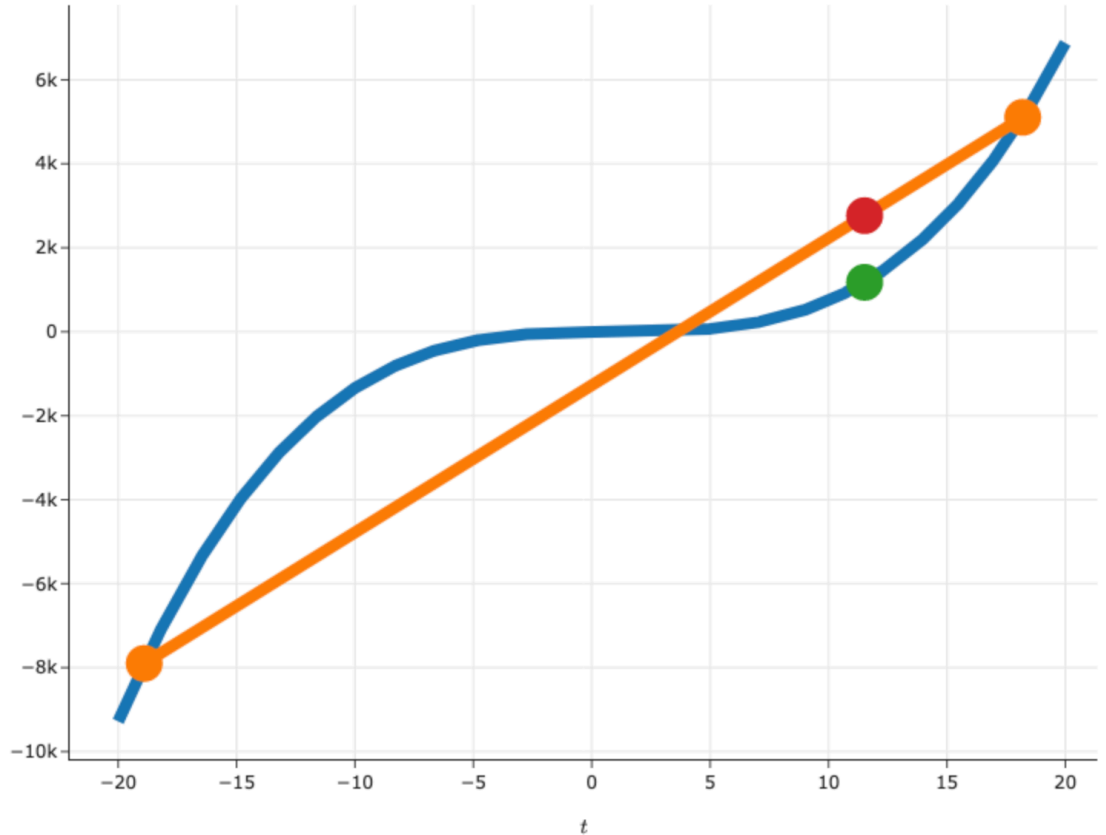


Formal definition of convexity

- A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **convex** if, for every a, b in the domain of f , and for every $t \in [0, 1]$:

$$(1 - t)f(a) + tf(b) \geq f((1 - t)a + tb)$$

- This is a formal way of restating the definition from the previous slide.



Question 🤔

Answer in chat

Which of these functions are **not** convex?

- A. $f(x) = |x|$.
- B. $f(x) = e^x$.
- C. $f(x) = \sqrt{x - 1}$.
- D. $f(x) = (x - 3)^{24}$.
- E. More than one of the above are non-convex.

Second derivative test for convexity

- If $f(t)$ is a function of a single variable and is **twice** differentiable, then $f(t)$ is convex if and only if:

$$\frac{d^2 f}{dt^2}(t) \geq 0, \quad \forall t$$

- Example: $f(x) = x^4$ is convex.

Why does convexity matter?

- Convex functions are (relatively) easy to minimize with gradient descent.
- **Theorem:** If $f(t)$ is convex and differentiable, then gradient descent converges to a **global minimum** of f , as long as the step size is small enough.
- **Why?**
 - Gradient descent converges when the derivative is 0.
 - For convex functions, the derivative is 0 only at one place – the global minimum.
 - In other words, if f is convex, gradient descent won't get "stuck" and terminate in places that aren't global minimums (local minimums, saddle points, etc.).

Nonconvex functions and gradient descent

- We say a function is **nonconvex** if it does not meet the criteria for convexity.
- Nonconvex functions are (relatively) difficult to minimize.
- Gradient descent **might** still work, but it's not guaranteed to find a global minimum.
 - We saw this at the start of the lecture, when trying to minimize
$$f(t) = 5t^4 - t^3 - 5t^2 + 2t - 9.$$

Choosing a step size in practice

- In practice, choosing a step size involves a lot of trial-and-error.
- In this class, we've only touched on "constant" step sizes, i.e. where α is a constant.

$$t_{i+1} = t_i - \alpha \frac{df}{dt}(t_i)$$

- **Remember:** α is the "step size", but the amount that our guess for t changes is $\alpha \frac{df}{dt}(t_i)$, not just α .
- In future courses, you'll learn about "decaying" step sizes, where the value of α decreases as the number of iterations increases.
 - Intuition: take much bigger steps at the start, and smaller steps as you progress, as you're likely getting closer to the minimum.

More examples

Example: Huber loss and the constant model

- First, we learned about squared loss,

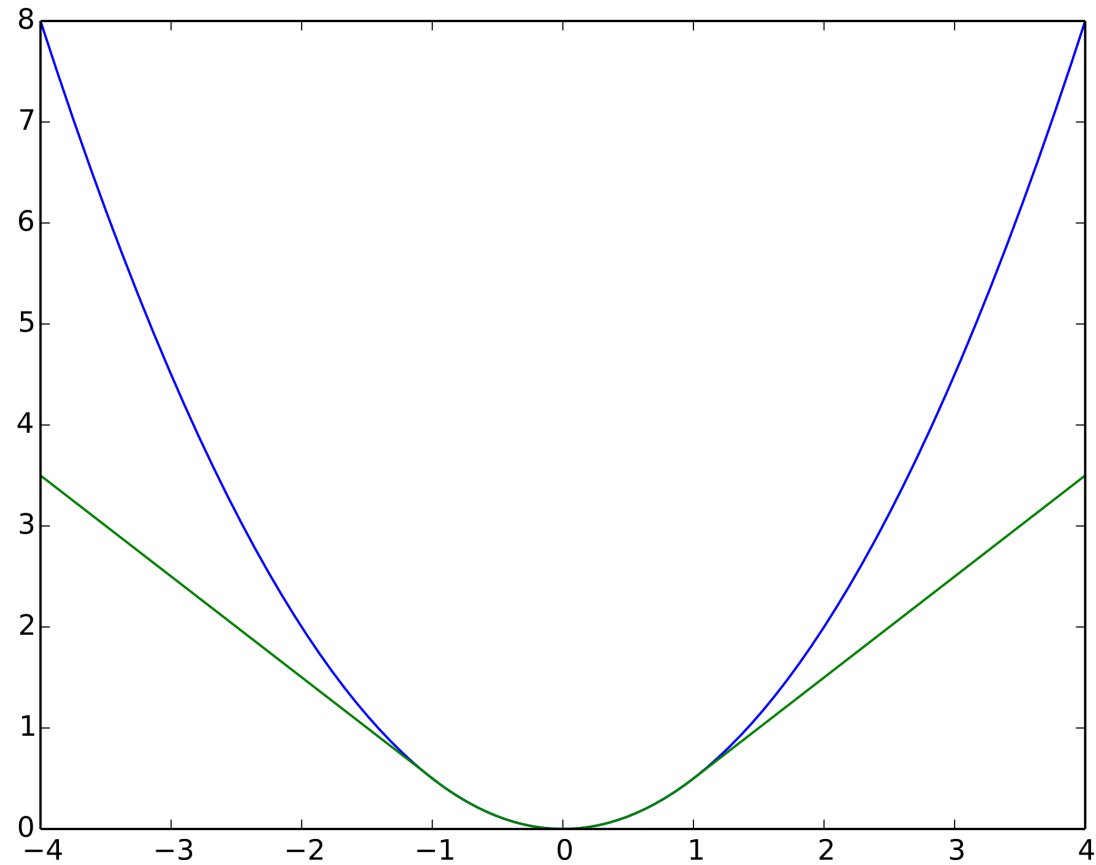
$$L_{\text{sq}}(y_i, H(x_i)) = (y_i - H(x_i))^2.$$

- Then, we learned about absolute loss,

$$L_{\text{abs}}(y_i, H(x_i)) = |y_i - H(x_i)|.$$

- Let's look at a new loss function, **Huber loss**:

$$L_{\text{huber}}(y_i, H(x_i)) = \begin{cases} \frac{1}{2} (y_i - H(x_i))^2 & \text{if } |y_i - H(x_i)| \leq \delta \\ \delta \cdot (|y_i - H(x_i)| - \frac{1}{2} \delta) & \text{otherwise} \end{cases}$$



Squared loss in blue, **Huber** loss in green.
Note that both loss functions are convex!

Minimizing average Huber loss for the constant model

- For the constant model, $H(x) = h$:

$$L_{\text{huber}}(y_i, h) = \begin{cases} \frac{1}{2}(y_i - h)^2 & \text{if } |y_i - h| \leq \delta \\ \delta \cdot (|y_i - h| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

$$\implies \frac{\partial L}{\partial h}(h) = \begin{cases} -(y_i - h) & \text{if } |y_i - h| \leq \delta \\ -\delta \cdot \text{sign}(y_i - h) & \text{otherwise} \end{cases}$$

- So, the **derivative** of empirical risk is:

$$\frac{dR_{\text{huber}}}{dh}(h) = \frac{1}{n} \sum_{i=1}^n \begin{cases} -(y_i - h) & \text{if } |y_i - h| \leq \delta \\ -\delta \cdot \text{sign}(y_i - h) & \text{otherwise} \end{cases}$$

- It's **impossible** to set $\frac{dR_{\text{huber}}}{dh}(h) = 0$ and solve by hand: we need gradient descent!

Let's try this out in practice! Follow along in [this notebook](#).

Minimizing functions of multiple variables

- Consider the function:

$$f(x_1, x_2) = (x_1 - 2)^2 + 2x_1 - (x_2 - 3)^2$$

- It has two **partial derivatives**: $\frac{\partial f}{\partial x_1}$ and $\frac{\partial f}{\partial x_2}$.

The gradient vector

- If $f(\vec{x})$ is a function of multiple variables, then its **gradient**, $\nabla f(\vec{x})$, is a vector containing its partial derivatives.
- Example:

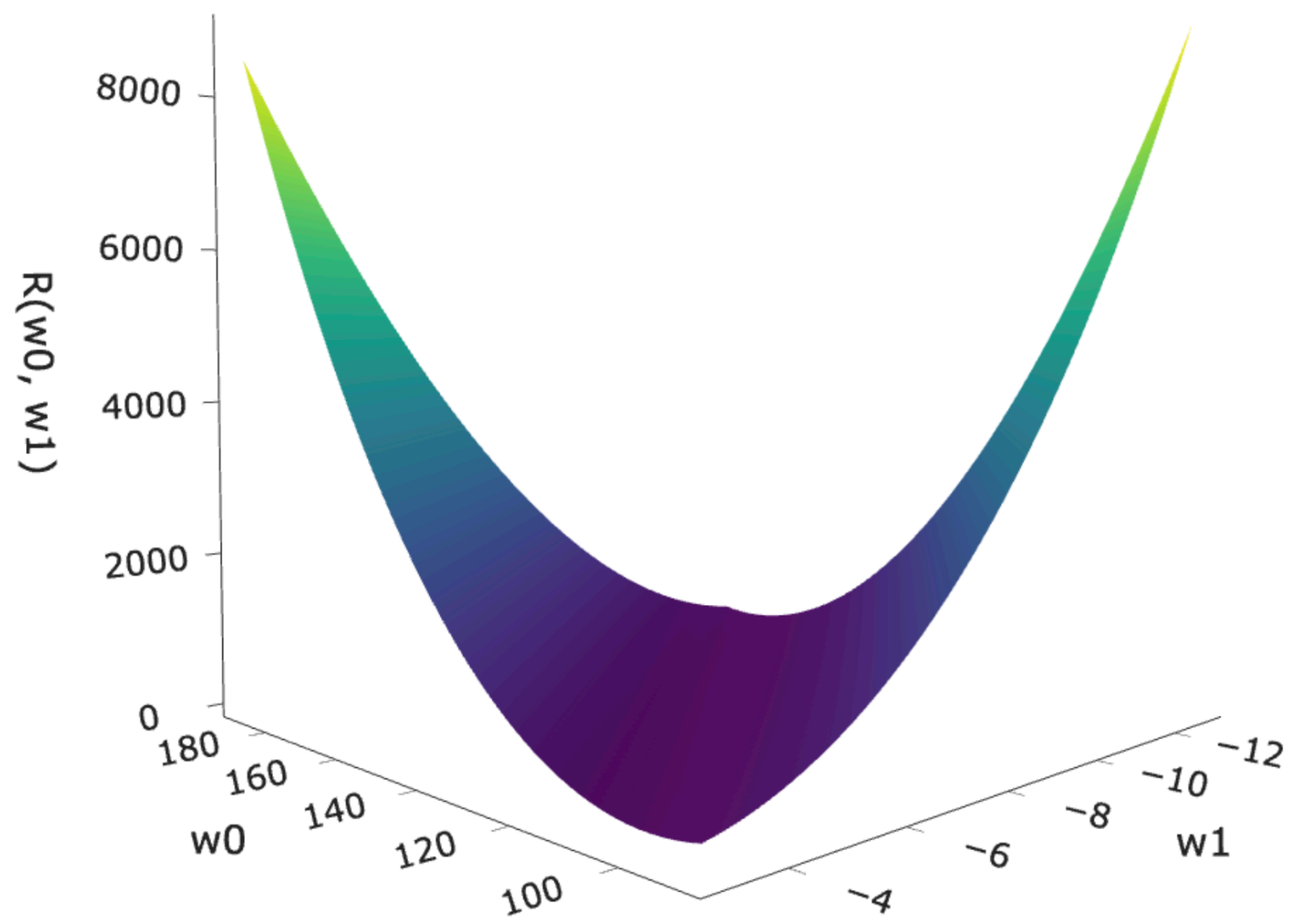
$$f(\vec{x}) = (x_1 - 2)^2 + 2x_1 - (x_2 - 3)^2$$

$$\nabla f(\vec{x}) = \begin{bmatrix} 2x_1 - 2 \\ -2x_2 + 6 \end{bmatrix}$$

- Example:

$$f(\vec{x}) = \vec{x}^T \vec{x}$$

$$\implies \nabla f(\vec{x}) =$$



Gradient descent for functions of multiple variables

- Example:

$$f(x_1, x_2) = (x_1 - 2)^2 + 2x_1 - (x_2 - 3)^2$$

$$\nabla f(\vec{x}) = \begin{bmatrix} 2x_1 - 2 \\ 2x_2 - 6 \end{bmatrix}$$

- The minimizer of f is a vector, $\vec{x}^* = \begin{bmatrix} x_1^* \\ x_2^* \end{bmatrix}$.
- We start with an initial guess, $\vec{x}^{(0)}$, and step size α , and update our guesses using:

$$\vec{x}^{(i+1)} = \vec{x}^{(i)} - \alpha \nabla f(\vec{x}^{(i)})$$

Exercise

$$f(x_1, x_2) = (x_1 - 2)^2 + 2x_1 - (x_2 - 3)^2$$

$$\nabla f(\vec{x}) = \begin{bmatrix} 2x_1 - 2 \\ 2x_2 - 6 \end{bmatrix}$$

$$\vec{x}^{(i+1)} = \vec{x}^{(i)} - \alpha \nabla f(\vec{x}^{(i)})$$

Given an initial guess of $\vec{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and a step size of $\alpha = \frac{1}{3}$, perform **two** iterations of gradient descent. What is $\vec{x}^{(2)}$?

Example: Gradient descent for simple linear regression

- To find optimal model parameters for the model $H(x) = w_0 + w_1x$ and squared loss, we minimized empirical risk:

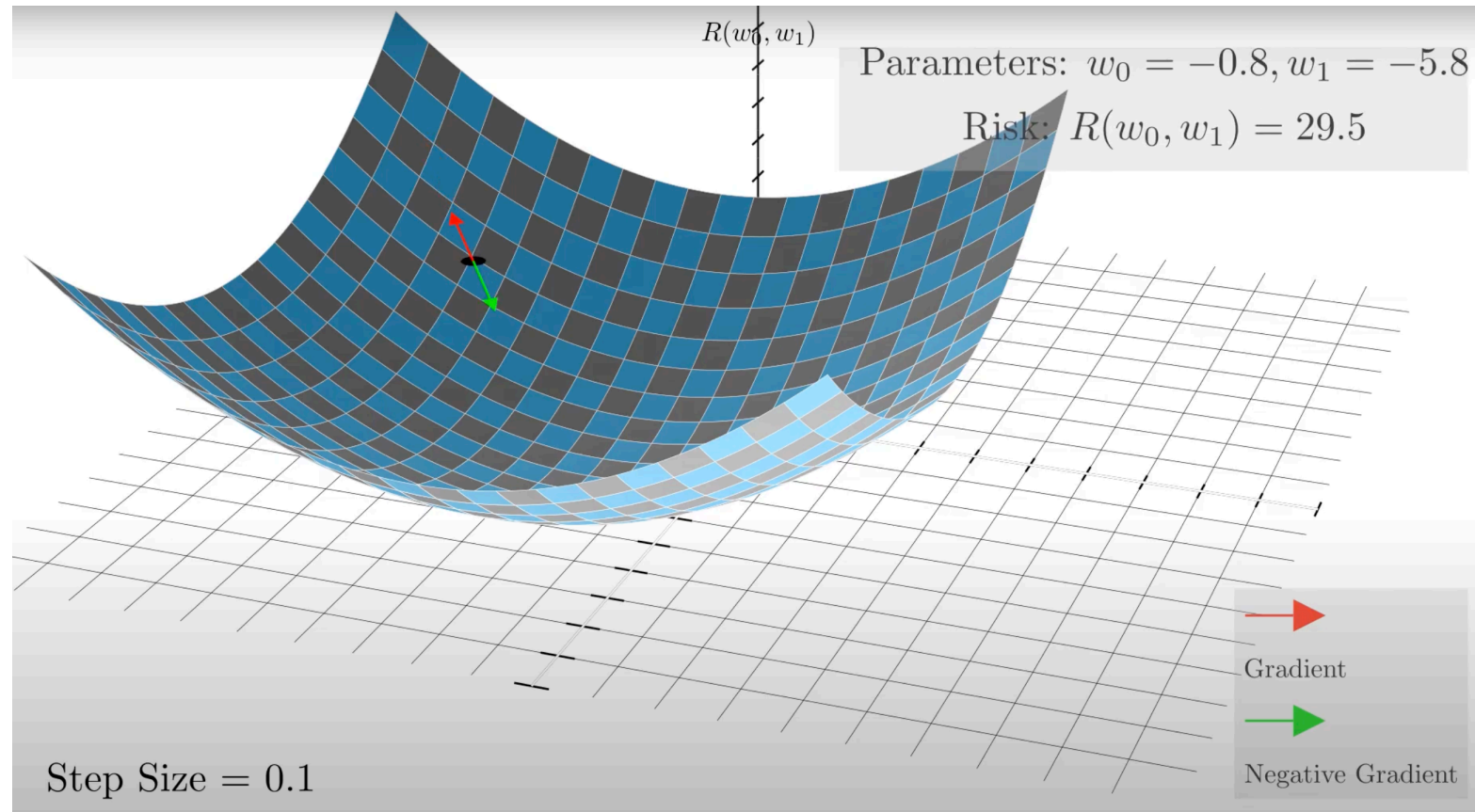
$$R_{\text{sq}}(w_0, w_1) = \frac{1}{n} \sum_{i=1}^n (y_i - (w_0 + w_1x_i))^2$$

- This is a function of multiple variables, and is differentiable, so it has a gradient!

$$\nabla R(\vec{w}) = \begin{bmatrix} -\frac{2}{n} \sum_{i=1}^n (y_i - (w_0 + w_1x_i)) \\ -\frac{2}{n} \sum_{i=1}^n (y_i - (w_0 + w_1x_i))x_i \end{bmatrix}$$

- **Key idea:** To find w_0^* and w_1^* , we *could* use gradient descent!

Gradient descent for simple linear regression, visualized



Let's watch  [this animation](#) that Jack made.

What's next?

- In Homework 4, you'll see a few questions involving today's material:
 - A question about convexity.
 - A question about implementing gradient descent to find optimal parameters for a model that is **not linear in its parameters**.
- On Monday next week after the midterm, we'll start talking about probability.